

Écosystème de Trading à Faible Latence

Architecture Hybride, Modélisation Stochastique et Deep Learning

Ce projet vise à concevoir un système de trading algorithmique orienté performance, capable d'opérer avec une latence très faible sur des marchés financiers fragmentés et volatils. Il repose sur une architecture hybride combinant un moteur d'exécution haute performance et une couche de recherche et de modélisation plus flexible.

1. Vue d'ensemble de l'architecture

Le système est structuré autour de deux composantes principales qui collaborent en temps réel :

Moteur d'exécution (C++) — Responsable de l'ingestion des flux de données de marché, de la tenue du carnet d'ordres, et de l'envoi des ordres vers les bourses. Cette partie est optimisée pour minimiser toute source de délai.

Couche de recherche et d'inférence (Python / LibTorch) — Chargée de l'entraînement des modèles de prédiction, de la modélisation du risque, et de la génération des signaux de trading. En production, l'inférence est intégrée directement dans le moteur d'exécution pour éviter tout aller-retour coûteux.

La communication entre ces deux environnements est conçue pour être aussi directe que possible, en évitant les protocoles réseau classiques au profit de mécanismes d'échange de données plus rapides.

2. Traitement des données de marché

Les données de marché arrivent en temps réel sous forme de flux continu. Le système les capture, les normalise et maintient une représentation à jour de l'état du marché — notamment le carnet d'ordres, qui recense l'ensemble des offres d'achat et de vente actives.

Cette étape est critique : la rapidité avec laquelle le système réagit à un changement de marché conditionne directement la qualité de l'exécution.

3. Génération de signaux par Deep Learning

Le système utilise des réseaux de neurones récurrents de type LSTM (Long Short-Term Memory) pour analyser l'évolution temporelle des données de marché et générer des prédictions directionnelles sur les prix.

Entraînement

La phase d'entraînement est réalisée hors ligne à partir de données historiques. Elle produit un modèle capable de détecter des motifs récurrents dans les séries temporelles financières — comme des variations de volatilité ou des déséquilibres dans le flux d'ordres.

Inférence en production

Une fois entraîné, le modèle est exporté dans un format compatible avec le moteur C++. En production, l'inférence s'effectue directement sans repasser par l'environnement Python, ce qui garantit une latence maîtrisée.

4. Modélisation stochastique et détection de régimes

Les marchés financiers ne sont pas stationnaires : ils alternent entre différents états — périodes calmes, tendances marquées, stress — qui influencent fortement la performance des stratégies.

Pour tenir compte de cette réalité, le système intègre des outils de modélisation stochastique permettant de caractériser la dynamique des prix, ainsi qu'un modèle probabiliste (Modèle de Markov Caché) capable d'identifier automatiquement le régime de marché courant.

Cette détection de régime permet d'ajuster dynamiquement le comportement du modèle LSTM : en cas de transition vers un environnement plus risqué, le système adapte ses seuils de décision en conséquence.

5. Gestion du risque

La gestion du risque est au cœur du système. Elle repose sur plusieurs mécanismes complémentaires :

Simulations de Monte Carlo — Le système génère un grand nombre de scénarios futurs plausibles pour évaluer les pertes potentielles du portefeuille et calculer des indicateurs comme la Value at Risk (VaR).

Dimensionnement des positions — La taille des ordres est déterminée en fonction de la performance attendue du signal et du niveau de volatilité actuel, afin de maintenir un budget de risque constant.

Vérifications pré-exécution — Avant tout envoi d'ordre, le système vérifie automatiquement les limites d'exposition, le débit d'ordres et la cohérence des prix pour se protéger contre les erreurs.

6. Flux opérationnel

Le pipeline de bout en bout peut se résumer ainsi : à la réception d'une nouvelle donnée de marché, le système met à jour son état interne, évalue le régime en cours, génère un signal de prédiction, en vérifie la cohérence avec les contraintes de risque, puis exécute l'ordre si toutes les conditions sont réunies. Chaque étape est conçue pour être la plus rapide possible, de la capture du tick à l'envoi de l'ordre.

Conclusion

Ce projet combine ingénierie logicielle haute performance et mathématiques financières avancées pour construire un écosystème de trading complet et robuste. L'accent est mis sur la cohérence de l'ensemble du pipeline — depuis la capture des données jusqu'à la gestion dynamique de l'exposition — afin de disposer d'un avantage technologique durable sur des marchés en constante évolution.